AI Security Findings

Overall risk: CRITICAL | Total findings: 1

Executive summary

Risk overview

Total findings: 1. Critical: 1, High: 0, Medium: 0, Low: 0.

Technical summary

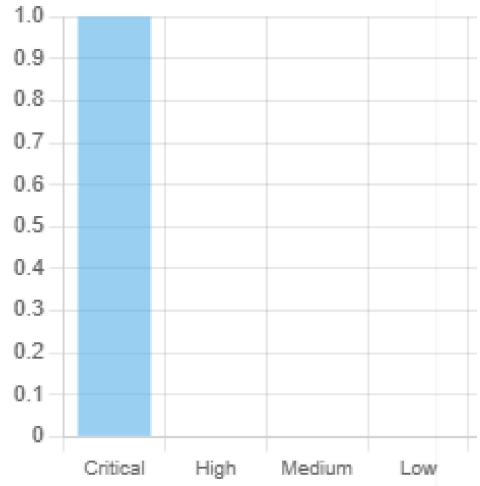
Overall Risk Level: Critical

Key Issues:
1. Prompt Injection Vulnerability – The source file contains embedded payload blocks in comments that may override system instructions, inject roles, and probe for template or SSRF vulnerabilities.
2. Data Exfiltration Risk – Crafted comments include commands that could be exploited to extract sensitive information such as system prompts, API keys, environment variables, and even system files like /etc/passwd.
3. Lack of Isolation and Sanitization – The current pipeline does not adequately isolate embedded instructions from executable code, potentially allowing attacker-supplied payloads to alter system behavior.

Quick Wins and Hardening Recommendations:
• Implement strict sandboxing for comment and payload processing, ensuring that any embedded instructions are treated as static text.
• Sanitize and validate all inputs and outputs of the pipeline to prevent execution of malicious payloads.
• Review and enforce robust filtering rules to separate analysis from the embedded instructions in source files.

This focused remediation effort should prioritize preventing code injection and information leakage by isolating unsanitized content, thereby fully mitigating the injection vectors identified.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
CRITICAL	Prompt Injection / LLM Hijacking	`Comments (entire payload block)`	The source file contains multiple embedded payload blocks in the comments that are designed to hijack the behavior of a multi-agent LLM pipeline. These payloads include direct instruction overrides, role injections, template injection probes, SSRF insights, and data exfiltration commands. While they are not executed as code, if the analysis pipeline or system prompt does not properly sanitize and isolate code from embedded instructions, an attacker might exploit them to reveal sensitive system data (such as system prompts, API keys, environment variables, or /etc/passwd content).	Ensure that the LLM pipeline or any processing system for source code treats all embedded instructions as inert comments. Use strict sandboxing and filtering to prevent any execution or leakage of sensitive data. Validate and sanitize any output derived from source files to make sure attacker-supplied payloads cannot alter system behavior or disclose confidential information.